

Prozesse, Threads und Tasks

Ausführungseinheiten, Ressourcen, Scheduling und Kernel-Sicht verstehen

2.06

Prozess, Thread und Task zeigen drei Blickwinkel

USER-SPACE-MODELL

Prozess

Ausführungs- und Ressourcenkontext mit eigener Thread-Gruppe und typischerweise eigenem Adressraum.

AUSFÜHRUNGSPFAD

Thread

Ein konkreter Kontrollfluss innerhalb eines Prozesses, mit eigenem Stack und eigener Thread-ID.

KERNEL-MODELL

Task

Die planbare Linux-Ausführungseinheit, repräsentiert durch eine eigene struct task_struct.

Ein Prozess kann mehrere Threads enthalten; jeder dieser Threads entspricht einem eigenen Kernel-Task.

Threads teilen Speicher, behalten eigenen Zustand

RESSOURCE	THREADS EINES PROZESSES	VERSCHIEDENE PROZESSE
Virtueller Adressraum	gemeinsam	üblicherweise getrennt
Dateideskriptor-Tabelle	typischerweise gemeinsam	eigene Tabellenkopie
Signal-Handler	gemeinsame Disposition	üblicherweise getrennt
Stack und Register	pro Thread getrennt	pro Task getrennt
TID und Signalmaske	pro Thread getrennt	pro Task getrennt

clone-Flags bestimmen das konkrete Teilen; bei fork werden Speicherseiten zunächst per Copy-on-Write genutzt.

task_struct hält den Zustand einer planbaren Einheit

VEREINFACHT · INCLUDE/LINUX/SCHED.H

```
struct task_struct {  
    unsigned int __state;  
    pid_t pid;  
    pid_t tgid;  
    int prio;  
    unsigned int policy;  
    struct mm_struct *mm;  
    struct sched_entity se;  
    /* weitere versionsabhängige Felder */  
};
```

01 Identität und Status

Task-ID,
Gruppenzugehörigkeit,
Laufzustand und
Scheduling-Informationen.

02 Ressourcenbezug

Verweise auf Adressraum,
Scheduling-Entity und
weitere gemeinsam
genutzte Strukturen.

Die Struktur gehört zu jedem Task; referenzierte Ressourcen können von mehreren Tasks geteilt werden.

TGID identifiziert den Prozess, TID den einzelnen Task

BEISPIEL · EIN PROZESS, DREI THREADS

AUSFÜHRUNG	PID / TGID	TID

Hauptthread	4242	4242
Worker A	4242	4243
Worker B	4242	4244


```
getpid() -> 4242 // Thread-Gruppe  
gettid() -> 4243 // konkreter Worker
```

01 Gruppen-ID

getpid() liefert innerhalb einer Thread-Gruppe für alle Threads denselben Wert.

02 Thread-ID

gettid() liefert die Linux-TID; pthread_self() liefert dagegen eine eigene POSIX-Kennung.

Beim Gruppenleiter gilt TID = TGID; andere Threads besitzen eigene TIDs bei unveränderter TGID.

fork und clone regeln die Ressourcenteilung

fork()

Neuer Prozess, neue Thread-Gruppe, eigener virtueller Adressraum

CLONE_VM

Adressraum wird zwischen erzeugenden und neuen Tasks geteilt

CLONE_FILES

Dateideskriptor-Tabelle wird gemeinsam verwendet

CLONE_SIGHAND

Signal-Handler werden geteilt; Signalmasken bleiben pro Thread

CLONE_THREAD

Neuer Task gehört zur bestehenden Thread-Gruppe

Erzeugung, Programmwechsel und Ende sind getrennt

01 fork oder pthread_create

Ein neuer Prozess beziehungsweise zusätzlicher Thread erhält eine eigene Kernel-Task-Struktur.

02 execve ersetzt das Programm

Der Prozess erhält ein neues Programmimage; execve erzeugt keinen zusätzlichen Prozess.

03 exit beendet die Ausführung

Threads und Prozesse geben Ressourcen kontrolliert frei; Prozessesstatus kann vorübergehend bleiben.

04 wait oder pthread_join räumen auf

Eltern übernehmen den Exit-Status; joinbare Threads werden über pthread_join abgeschlossen.

Ein Zombie läuft nicht: Er bewahrt lediglich Exit-Informationen, bis der Elternprozess sie übernimmt.

Task-Zustände erklären, warum Arbeit gerade nicht läuft

R · TASK_RUNNING

Ausführbar oder aktuell auf einer CPU aktiv

S · INTERRUPTIBLE

Unterbrechbarer Schlaf; ein Signal kann das Warten beenden

D · UNINTERRUPTIBLE

Nicht normal durch Signale unterbrechbarer Wartezustand

T · STOPPED / TRACED

Task ist angehalten oder wird durch einen Tracer kontrolliert

Z · EXIT_ZOMBIE

Beendet; Exit-Status wurde noch nicht vollständig übernommen

Der Scheduler wählt Tasks, nicht ganze Prozesse

RUNNABLE TASKS

Einzeln planbar

Jeder ausführbare Thread besitzt eigene Scheduler-Daten und kann unabhängig CPU-Zeit erhalten.

MEHRERE CPUS

Parallel möglich

Threads desselben Prozesses können gleichzeitig auf unterschiedlichen CPU-Kernen laufen.

AKTUELLER FAIR-PFAD

CFS → EEVDF

Neuere Kernel wechseln von klassischem CFS schrittweise zu Earliest Eligible Virtual Deadline First.

EEVDF berücksichtigt Fairness und virtuelle Deadlines; tatsächliches Verhalten hängt von der Kernelversion ab.

Scheduling-Policies setzen unterschiedliche Prioritäten

SCHED_OTHER / NORMAL

Allgemeine Anwendungen und faire CPU-Zuteilung

SCHED_BATCH / IDLE

Batch-Jobs beziehungsweise besonders niedrige Priorität

SCHED_FIFO / RR

Echtzeit-Policies; RR ergänzt Zeitscheiben

SCHED_DEADLINE

Laufzeit, Deadline und Periode explizit beschrieben

Echtzeitprioritäten benötigen besondere Berechtigungen und können das System bei falscher Nutzung blockieren.

Kontextwechsel tauschen den aktiven Task

01 Vorherigen Task sichern

Register, Instruktionszeiger, Stack-Kontext und Scheduling-Informationen festhalten.

02 Nächsten Task auswählen

Policy, Priorität, Fairness, CPU-Affinität und verfügbare CPUs berücksichtigen.

03 Adressraum bei Bedarf wechseln

Zwischen Threads derselben Gruppe kann derselbe mm-Kontext weiterverwendet werden.

04 Ausführung wieder aufnehmen

Der ausgewählte Task läuft mit eigenem Stack und eigenen Registern weiter.

Thread-Wechsel können gemeinsamen Speicher nutzen; Cache-, TLB- und Scheduling-Kosten bleiben relevant.

Kernel-Threads arbeiten ohne User-Space-Programm

KTHREADS · WORKQUEUES · CPU-WORKER

```
# Beispielhafte Kernel-Threads in ps:  
[kthreadd]  
[kworker/0:1]  
[migration/0]  
  
# Kernel-intern:  
worker = kthread_run(worker_fn, data, "demo");  
  
# kworker führt Workqueue-Aufträge aus.
```

01 Kernel-Kontext

Kernel-Threads besitzen einen Task und werden geplant, führen aber kein gewöhnliches User-Programm aus.

02 Workqueues

kworker verarbeitet Work-Items; nicht jeder einzelne Auftrag erzeugt einen neuen Thread.

kthread_run erstellt und aktiviert einen Kernel-Thread; Namen und Sichtbarkeit hängen vom System ab.

Gemeinsamer Speicher verlangt gezielte Synchronisation

GETEILTE DATEN

Race Conditions

Mehrere Threads können denselben Speicher gleichzeitig lesen und schreiben.

KOORDINATION

Mutex und Signale

Mutex, Condition Variable, atomare Operationen oder Join schützen passende Übergänge.

ENTWURFSFEHLER

Deadlock und Blockade

Falsche Lock-Reihenfolgen, wartende Threads oder unangemessene Prioritäten verhindern Fortschritt.

Parallelität bedeutet nicht automatisch mehr Durchsatz: Synchronisation, Wartezeit und Datenzugriffe zählen.

C-Beispiel: Gleiche PID und unterschiedliche TIDs

DEMO.C · LINUX / GLIBC MIT GETTID()

```
#define _GNU_SOURCE
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

static void *worker(void *arg) {
    printf("worker PID=%d TID=%d\n", getpid(), gettid());
    sleep(20);
    return NULL;
}

int main(void) {
    pthread_t thread;
    printf("main PID=%d TID=%d\n", getpid(), gettid());
    pthread_create(&thread, NULL, worker, NULL);
    pthread_join(thread, NULL);
}
```

Erwartung: Haupt- und Worker-Thread zeigen dieselbe PID, aber unterschiedliche Linux-Thread-IDs.

Prozesse und Threads direkt im System prüfen

LABOR · WÄHREND DER WORKER NOCH LÄUFT

```
$ cc -Wall -Wextra -pthread demo.c -o demo
$ ./demo &
$ target_pid=$!

$ ps -L -p "$target_pid" -o pid,tid,ppid,psr,stat,comm
$ ls "/proc/$target_pid/task"
$ awk '/^(Name|Tgid|Pid|Threads):/' "/proc/$target_pid/status"
$ wait "$target_pid"
```

01 ps pro Thread

-L zeigt beide Ausführungseinheiten mit gemeinsamer PID und individuellen TIDs.

02 procfs prüfen

task/ enthält je Thread ein TID-Verzeichnis; status nennt TGID und Thread-Anzahl.

Damit wird das Kernel-Modell sichtbar: eine Thread-Gruppe, mehrere Tasks und getrennte Scheduler-Einheiten.